

***** main.zig

```

const rl = @import("raylib");
const std = @import("std");
const print = std.debug.print;

const sqlite3 = @import("lib/Database.zig");
const gamemap = @import("lib/GameMap.zig");
const tile = @import("lib/Tile.zig");
const terrain = @import("lib/Terrain.zig");
const hexagon = @import("lib/Hexagon.zig");
const asset = @import("lib/Asset.zig");
const card = @import("lib/Card.zig");
const ui = @import("lib/UI.zig");

var GameFlags: u64 = 0;
//var GameFlags_prev: u64 = 0;
var hexLoc = [3]i32{ 0, 0, 0 };
var mousePos: rl.Vector2 = undefined;

pub fn main() !void {
    const db = sqlite3.Database.init();
    defer db.close();

    var arena = std.heap.ArenaAllocator.init(std.heap.page_allocator);
    defer arena.deinit();
    const allocator = arena.allocator();

    // *****
    var Textures = std.ArrayList(rl.Texture).empty;
    defer Textures.deinit(allocator);

    var Tiles = std.ArrayList(tile.Tile).empty;
    defer Tiles.deinit(allocator);

    // ***** Terrains
    var Hills = std.ArrayList(terrain.Hill).empty;
    defer Hills.deinit(allocator);

    var Lakes = std.ArrayList(terrain.Lake).empty;
    defer Lakes.deinit(allocator);

    var Rivers = std.ArrayList(terrain.River).empty;
    defer Rivers.deinit(allocator);

    var Roads = std.ArrayList(terrain.Road).empty;
    defer Roads.deinit(allocator);

    var WholeHex = std.ArrayList(terrain.WholeHex).empty;
    defer WholeHex.deinit(allocator);

    var Bridges = std.ArrayList(terrain.Bridge).empty;
    defer Bridges.deinit(allocator);

    var Tunnel_Entrances = std.ArrayList(terrain.Tunnel_Entrance).empty;
    defer Tunnel_Entrances.deinit(allocator);

    // ***** Path
    var Paths = std.ArrayList(terrain.Path).empty;
    defer Paths.deinit(allocator);

    const A = hexagon.Hexagon.init(0, 0);
    const B = hexagon.Hexagon.init(5, 8);
    try A.get_path(allocator, B, &Paths);

    // ***** Asset
    var Imgs = std.ArrayList(rl.Texture).empty;
    defer Imgs.deinit(allocator);

    var ImgIDs = std.ArrayList(asset.ImgID2index).empty;
    defer ImgIDs.deinit(allocator);

    var Combatants = std.ArrayList(asset.Combatant).empty;
    defer Combatants.deinit(allocator);

    var Chosen_Units_in_Hex = std.ArrayList(asset.Combatant).empty;
    defer Chosen_Units_in_Hex.deinit(allocator);

    // *****
    const pxX = db.get_float_vals("pxX");
    const pxY = db.get_float_vals("pxY");
    const hex_width = pxX[0];
    const hex_height = pxY[0];

    // *****
    const windowWidth = 1280;
    const windowHeight = 720;

    rl.setConfigFlags({ .window_undecorated = true });
    rl.initWindow(windowWidth, windowHeight, "The Long Road");
    defer rl.closeWindow();

    //rl.toggleFullscreen();

    // *****
    try db.add_map_tiles(allocator, &Textures, &Tiles);

    const tileLetters = try db.get_tileLetters(allocator);
    defer allocator.free(tileLetters);

    var gMap = gamemap.GameMap.init(tileLetters, &Tiles);
    gMap.status(&Tiles);

    // =====

```

```

try db.add_map_hills(allocator, &Hills);
try db.add_map_lakes(allocator, &Lakes);
try db.add_map_rivers(allocator, &Rivers);
try db.add_map_roads(allocator, &Roads);
try db.add_map_bridges(allocator, &Bridges);
try db.add_map_tunnel_entrances(allocator, &Tunnel_Entrances);

try db.add_map_rollings(allocator, &WholeHex);
try db.add_map_cultivated(allocator, &WholeHex);
try db.add_map_forests(allocator, &WholeHex);
try db.add_map_cities(allocator, &WholeHex);
try db.add_map_towns(allocator, &WholeHex);

try db.add_map_combatants(allocator, &Imgs, &ImgIDs, &Combatants);

// =====
//const card_pac = try card.PlayerAid.init(-1509, 0, "TLR/Player Aid Card.png", "TLR/Last Turn.png", db);
//const card_pdw = try card.Card.init(0, -1417, "TLR/TLR_Para_Deck_Window.png");
//const card_compass = try card.Card.init(-163, -163, "TLR/LAR_Compass.png");
//const card_phb = try card.Card.init(438, 0, 90.0, "TLR/TLR_Player_Hand_Background.png");

// *****
if (terrain.is_hill_blocks_L05(&Hills, &Paths)) {
    print("***** block *****\n", .{});
}

// *****
// *****
// *****
rl.setTargetFPS(12);

var camera = rl.Camera2D{
    .target = .{ .x = 0, .y = 0 },
    .offset = .{ .x = 0, .y = 0 },
    .zoom = 1.0,
    .rotation = 0,
};

var toggle: i32 = 1;
while (!rl.windowShouldClose()) {
    if (rl.isKeyPressed(.space)) {
        if (toggle == 0) {
            for (0..Combatants.items.len) |i| {
                if (Combatants.items.ptr[i].instanceID == 1001) {
                    Combatants.items.ptr[i].is_visible = true;
                }
            }
            toggle = 1;
        } else {
            for (0..Combatants.items.len) |i| {
                if (Combatants.items.ptr[i].instanceID == 1001) {
                    Combatants.items.ptr[i].is_visible = false;
                }
            }
            toggle = 0;
        }
    }

    // Translate based on mouse right click
    if (rl.isMouseButtonDown(.right)) {
        var delta = rl.getMouseDelta();
        delta = rl.math.vector2Scale(delta, -1.0 / camera.zoom);
        camera.target = rl.math.vector2Add(camera.target, delta);
    }

    if (rl.isMouseButtonDown(.middle)) {
        //if (GameFlags != GameFlags_prev) {
        print("reset\n", .{});
        GameFlags = 0;
        Chosen_Units_in_Hex.clearAndFree(allocator);
        // GameFlags_prev = 0;
        //}
    }

    if (rl.isMouseButtonDown(.left)) {
        if ((GameFlags & (1 << 0)) == 0) { // set selected mode
            print("set selected mode\n", .{});
            //GameFlags_prev = GameFlags;
            GameFlags |= (1 << 0);
        }
    }

    // Zoom based on mouse wheel
    const wheel = rl.getMouseWheelMove();
    if (wheel != 0) {
        const mouseWorldPos = rl.getScreenToWorld2D(rl.getMousePosition(), camera);
        camera.offset = rl.getMousePosition();
        camera.target = mouseWorldPos;

        // Zoom increment
        var scaleFactor = 1.0 + (0.25 * @abs(wheel));
        if (wheel < 0) {
            scaleFactor = 1.0 / scaleFactor;
        }
        camera.zoom = rl.math.clamp(camera.zoom * scaleFactor, 0.125, 64.0);
    }

    // =====
    rl.beginDrawing();
    defer rl.endDrawing();

    rl.clearBackground(.black);
}

```

```

camera.begin();
defer camera.end();

// ***** map tiles
for (0..4) |row| {
    for (0..4) |col| {
        if (gMap.GMap[@intCast(col)][@intCast(row)] >= 0) {
            const tile_num = Tiles.items.ptr[@intCast(gMap.GMap[@intCast(col)][@intCast(row)])].index;
            const rotation = Tiles.items.ptr[@intCast(gMap.GMap[@intCast(col)][@intCast(row)])].rotation;

            const X = @as(f32, @floatFromInt(col)) * @as(f32, @floatFromInt(db.pixelCount[0]));
            const Y = @as(f32, @floatFromInt(row)) * @as(f32, @floatFromInt(db.pixelCount[1]));

            if (rotation > 0) {
                const mod_x = X + @as(f32, @floatFromInt(db.pixelCount[0]));
                const mod_y = Y + @as(f32, @floatFromInt(db.pixelCount[1]));
                rl.drawTextureEx(Textures.items.ptr[@intCast(tile_num)], .{ .x = mod_x, .y = mod_y }, rotation, 1.0, .white);
            } else {
                rl.drawTextureEx(Textures.items.ptr[@intCast(tile_num)], .{ .x = X, .y = Y }, rotation, 1.0, .white);
            }
        }
    }
}

// units
for (0..Combatants.items.len) |i| {
    Combatants.items.ptr[i].drawMe(hex_width, hex_height, &Imgs);
}

if ((GameFlags & (1 << 0)) == 1) try ui.mode_1(&GameFlags, camera, hex_width, hex_height, &Combatants, &Chosen_Units_in_Hex,
allocator);
} // camera block

if (Chosen_Units_in_Hex.items.len > 0) {
    rl.drawTexture(Imgs.items.ptr[@intCast(Chosen_Units_in_Hex.items.ptr[0].imgIndex)], 0, 0, .white);
}
} // Game loop
}

```

***** Asset.zig

```

const rl = @import("raylib");
const std = @import("std");
const print = std.debug.print;

pub const ImgID2index = struct {
    imgID: i32,

    pub fn init(imgID: i32) ImgID2index {
        return ImgID2index{
            .imgID = imgID,
        };
    }
};

pub const Combatant = struct {
    instanceID: i32,
    hex_x: i32,
    hex_y: i32,
    hex_z: i32,
    flag1: i32,
    id: i32, // Combatant ID
    currState: i32, // img ID
    imgIndex: i32,
    is_visible: bool,
    descrip: []const u8,
    stack_lvl: i32,
    // imagefile: [64]u8,
    // imagefile_len: i32,

    pub fn init(instanceID: i32, hex_x: i32, hex_y: i32, id: i32, currState: i32, descrip: []const u8, stack_lvl: i32) Combatant {
        return Combatant{
            .instanceID = instanceID,
            .hex_x = hex_x,
            .hex_y = hex_y,
            .hex_z = 0,
            .flag1 = 0,
            .id = id,
            .currState = currState,
            .imgIndex = 0,
            .is_visible = true,
            .descrip = descrip,
            .stack_lvl = stack_lvl,
            // .imagefile = undefined,
            // .imagefile_len = 0,
        };
    }
};

pub fn drawMe(self: Combatant, hex_width: f32, hex_height: f32, img: *std.ArrayList(rl.Texture)) void {
    if (!self.is_visible) return;

    const adjX: f32 = 79.0;
    const adjY: f32 = hex_height * 0.84;
    const adjYa: f32 = hex_height * 0.35;

    var X: f32 = @floatFromInt(self.hex_x);
    X = (X * hex_width) - adjX;
    var Y: f32 = @floatFromInt(self.hex_y);
    if (@mod(self.hex_x, 2) == 0) {
        Y = (Y * hex_height) - adjYa;
    } else {
        Y = (Y * hex_height) - adjY;
    }
}

```

```

const aF32: f32 = @floatFromInt(self.stack_lvl);
X += (aF32 * 8.0);
Y -= (aF32 * 8.0);

    rl.drawTextureEx(img.items.ptr[@intCast(self.imgIndex)], rl.Vector2.init(X, Y), 0.0, 1.0, .white);
}
};

***** Card.zig
const rl = @import("raylib");
const std = @import("std");
const print = std.debug.print;

const sqlite3 = @import("Database.zig");

pub const Card = struct {
    pxX: i32,
    pxY: i32,
    texture: rl.Texture,
    rotation: f32,

    // *****
    pub fn init(pxX: i32, pxY: i32, rotation: f32, fname: []const u8) !Card {
        var buff: [60]u8 = undefined;
        const filename = try std.fmt.bufPrintZ(&buff, "{s}", .{fname});
        const texture = try rl.loadTexture(filename);

        return Card{
            .pxX = pxX,
            .pxY = pxY,
            .texture = texture,
            .rotation = rotation,
        };
    }
};

pub const PlayerAid = struct {
    pxX: i32,
    pxY: i32,
    plate: rl.Texture,
    marker: rl.Texture,
    mrk_X: i32,
    mrk_Y: i32,
    curr_turn: i32,

    // *****
    pub fn init(pxX: i32, pxY: i32, fname: []const u8, m: []const u8, db: sqlite3.Database) !PlayerAid {
        var buff: [60]u8 = undefined;

        const filename = try std.fmt.bufPrintZ(&buff, "{s}", .{fname});
        const plate = try rl.loadTexture(filename);

        const mrk = try std.fmt.bufPrintZ(&buff, "{s}", .{m});
        const marker = try rl.loadTexture(mrk);

        var turns: i32 = db.get_int_vals_session("turns").@"0";
        turns -= 1;

        const upper_left: [2]i32 = [_]i32{ 17, 333 };
        const grid_dim: [2]i32 = [_]i32{ 978, 781 };
        const rect_dim: [2]i32 = [_]i32{ @divTrunc(grid_dim[0], 5), @divTrunc(grid_dim[1], 4) };

        const turn_X: i32 = @mod(turns, 5);
        const turn_Y: i32 = @divTrunc(turns, 4);

        const grid_X: i32 = rect_dim[0] * turn_X;
        const grid_Y: i32 = rect_dim[1] * turn_Y;

        const loc_X: i32 = grid_X + upper_left[0];
        const loc_Y: i32 = grid_Y + upper_left[1];

        const card_dim: [2]i32 = [_]i32{ 1504, 1131 };
        const adj_X: i32 = loc_X - card_dim[0] + 15;
        const adj_Y: i32 = loc_Y + pxY + 18;

        return PlayerAid{
            .pxX = pxX,
            .pxY = pxY,
            .plate = plate,
            .marker = marker,
            .mrk_X = adj_X,
            .mrk_Y = adj_Y,
            .curr_turn = 1,
        };
    }
};

***** Database.zig
const rl = @import("raylib");
const std = @import("std");
const print = std.debug.print;

const tile = @import("Tile.zig");
const terrain = @import("Terrain.zig");
const asset = @import("Asset.zig");

const c = @cImport({
    @cInclude("sqlite3.h");
});

pub const Database = struct {
    db: ?*c.sqlite3, // in-memory Database

```

```

currSession: i64,
pixelCount: [2]i32,

// *****
pub fn init() Database {
    var db: ?c.sqlite3 = null;

    // Open the databases
    if (c.sqlite3_open("DB/TLR.db", &db) != c.SQLITE_OK) {
        print("Error opening database\n", .{});
    }
    defer _ = c.sqlite3_close(db); // close disk connection after init.

    // Prepare the SQL statement
    var stmt: ?c.sqlite3_stmt = null;
    const sql = "SELECT val_int0 FROM GameMeta WHERE attrib = 'currSession'";
    _ = c.sqlite3_prepare_v2(db, sql, -1, &stmt, null);
    defer _ = c.sqlite3_finalize(stmt);

    // Evaluate the statement
    var currSession: i64 = 0;
    if (c.sqlite3_step(stmt) == c.SQLITE_ROW) {
        currSession = c.sqlite3_column_int64(stmt, 0);
    }

    // Prepare the SQL statement
    const sql_1 = "SELECT val_int0, val_int1 FROM GameMeta WHERE attrib = 'pixelCount'";
    _ = c.sqlite3_prepare_v2(db, sql_1, -1, &stmt, null);

    // Evaluate the statement
    var i64_X: i64 = 0;
    var i64_Y: i64 = 0;
    if (c.sqlite3_step(stmt) == c.SQLITE_ROW) {
        i64_X = c.sqlite3_column_int64(stmt, 0);
        i64_Y = c.sqlite3_column_int64(stmt, 1);
    }

    const curS: i32 = @intCast(currSession);

    // -----
    // Prepare the SQL statement
    const sql_3 =
        \DELETE FROM GameImg
        \WHERE sessionID = ?1
    ;
    _ = c.sqlite3_prepare_v2(db, sql_3, -1, &stmt, null);

    // Binding
    _ = c.sqlite3_bind_int(stmt, 1, curS);

    // Execute statement
    _ = c.sqlite3_step(stmt);

    // -----
    // Prepare the SQL statement
    const sql_2 =
        \INSERT INTO GameImg (id, state_num, imgID, sessionID)
        \SELECT id, 0, state0, ?1 FROM COMBATANT WHERE id in
        \((
        \SELECT distinct(id) FROM GameCombatant
        \WHERE sessionID = ?1
        \))
        \UNION
        \SELECT id, 1, state1, ?1 FROM COMBATANT WHERE id in
        \((
        \SELECT distinct(id) FROM GameCombatant
        \WHERE sessionID = ?1
        \))
        \UNION
        \SELECT id, 2, state2, ?1 FROM COMBATANT WHERE id in
        \((
        \SELECT distinct(id) FROM GameCombatant
        \WHERE sessionID = ?1
        \))
        \UNION
        \SELECT id, 3, state3, ?1 FROM COMBATANT WHERE id in
        \((
        \SELECT distinct(id) FROM GameCombatant
        \WHERE sessionID = ?1
        \))
        \UNION
        \SELECT id, 4, wreck, ?1 FROM COMBATANT WHERE id in
        \((
        \SELECT distinct(id) FROM GameCombatant
        \WHERE sessionID = ?1
        \))
    ;
    _ = c.sqlite3_prepare_v2(db, sql_2, -1, &stmt, null);

    // Binding
    _ = c.sqlite3_bind_int(stmt, 1, curS);

    // Execute statement
    _ = c.sqlite3_step(stmt);

    // -----
    // Prepare the SQL statement
    const sql_4 =
        \DELETE FROM GameAsset
        \WHERE sessionID = ?1
    ;
    _ = c.sqlite3_prepare_v2(db, sql_4, -1, &stmt, null);

    // Binding

```

```

_ = c.sqlite3_bind_int(stmt, 1, curS);

// Execute statement
_ = c.sqlite3_step(stmt);

// -----
// Prepare the SQL statement
const sql_5 =
    "\\INSERT INTO GameAsset (imgID, filename, sessionID)
    \\SELECT DISTINCT(imgID), file, sessionID FROM v_gameimg
    \\WHERE sessionID = ?1
;
_ = c.sqlite3_prepare_v2(db, sql_5, -1, &stmt, null);

// Binding
_ = c.sqlite3_bind_int(stmt, 1, curS);

// Execute statement
_ = c.sqlite3_step(stmt);

// =====
return Database{
    .db = db,
    .currSession = currSession,
    .pixelCount = @{ @intCast(i64_X), @intCast(i64_Y) },
};
}

// *****
pub fn add_map_combatants(self: Database, allocator: std.mem.Allocator, img: *std.ArrayList(r1.Texture), imgid:
*std.ArrayList(asset.ImageID2index), cmb: *std.ArrayList(asset.Combatant)) !void {
    const curS: i32 = @intCast(self.currSession);

    // Prepare the SQL statement
    var stmt: ?*c.sqlite3_stmt = null;
    const sql =
        "\\SELECT imgID, filename FROM GameAsset
        \\WHERE
        \\sessionID = ?1
    ;
    _ = c.sqlite3_prepare_v2(self.db, sql, -1, &stmt, null);
    defer _ = c.sqlite3_finalize(stmt);

    // Binding
    _ = c.sqlite3_bind_int(stmt, 1, curS);

    // Evaluate the statement
    while (c.sqlite3_step(stmt) == c.SQLITE_ROW) {
        const imgID = c.sqlite3_column_int(stmt, 0);
        const filename = std.mem.span(c.sqlite3_column_text(stmt, 1));
        const fname = try allocator.alloc(u8, filename.len + 4);
        defer allocator.free(fname);
        @memcpy(fname[0..4], "TLR/");
        @memcpy(fname[4..], filename);

        const c_str = try allocator.dupeZ(u8, fname);
        defer allocator.free(c_str);
        _ = try img.append(allocator, try r1.loadTexture(c_str));

        const aImgID2index = asset.ImageID2index.init(imgID);
        _ = try imgid.append(allocator, aImgID2index);
    }

    // -----
    const sql_1 =
        "\\SELECT instanceID, hex_x, hex_y, id, currState, descrip, stack
        \\FROM v_gamecombatant
        \\WHERE
        \\sessionID = ?1
        \\ORDER BY hex_x, hex_y, stack
    ;
    _ = c.sqlite3_prepare_v2(self.db, sql_1, -1, &stmt, null);

    // Binding
    _ = c.sqlite3_bind_int(stmt, 1, curS);

    // Evaluate the statement
    while (c.sqlite3_step(stmt) == c.SQLITE_ROW) {
        const instanceID = c.sqlite3_column_int(stmt, 0);
        const hex_x = c.sqlite3_column_int(stmt, 1);
        const hex_y = c.sqlite3_column_int(stmt, 2);
        const id = c.sqlite3_column_int(stmt, 3);
        const currState = c.sqlite3_column_int(stmt, 4);
        const stack_lvl = c.sqlite3_column_int(stmt, 6);

        const raw_str = std.mem.span(c.sqlite3_column_text(stmt, 5));
        const descrip = try allocator.alloc(u8, raw_str.len);
        //defer allocator.free(descrip);
        @memcpy(descrip[0..], raw_str);

        //const c_str = try allocator.dupeZ(u8, descrip);
        //const cbt = asset.Combatant.init(instanceID, hex_x, hex_y, id, currState, c_str);
        const cbt = asset.Combatant.init(instanceID, hex_x, hex_y, id, currState, descrip, stack_lvl);

        _ = try cmb.append(allocator, cbt);
    }

    // -----
    // defining imgIndex
    for (0..cmb.items.len) |i| {
        var count: i32 = 0;
        for (0..imgid.items.len) |j| {
            if (cmb.items.ptr[i].currState == imgid.items.ptr[j].imgID) {
                cmb.items.ptr[i].imgIndex = count;
            }
        }
    }
}

```

```

        break;
    }
    count += 1;
}
}
}

// *****
pub fn add_map_towns(self: Database, allocator: std.mem.Allocator, wh: *std.ArrayList(terrain.WholeHex)) !void {
    // Prepare the SQL statement
    var stmt: ?*c.sqlite3_stmt = null;
    const sql =
        \\SELECT hex_x, hex_y FROM GameMap
        \\WHERE
        \\terrainNum = 11 AND
        \\sessionID = ?1
    ;
    _ = c.sqlite3_prepare_v2(self.db, sql, -1, &stmt, null);
    defer _ = c.sqlite3_finalize(stmt);

    // Binding
    const curS: i32 = @intCast(self.currSession);
    _ = c.sqlite3_bind_int(stmt, 1, curS);

    // Evaluate the statement
    while (c.sqlite3_step(stmt) == c.SQLITE_ROW) {
        const hex_x = c.sqlite3_column_int(stmt, 0);
        const hex_y = c.sqlite3_column_int(stmt, 1);

        const aTown = terrain.WholeHex.init(hex_x, hex_y, 11);
        _ = try wh.append(allocator, aTown);
    }
}

// *****
pub fn add_map_cities(self: Database, allocator: std.mem.Allocator, wh: *std.ArrayList(terrain.WholeHex)) !void {
    // Prepare the SQL statement
    var stmt: ?*c.sqlite3_stmt = null;
    const sql =
        \\SELECT hex_x, hex_y FROM GameMap
        \\WHERE
        \\terrainNum = 2 AND
        \\sessionID = ?1
    ;
    _ = c.sqlite3_prepare_v2(self.db, sql, -1, &stmt, null);
    defer _ = c.sqlite3_finalize(stmt);

    // Binding
    const curS: i32 = @intCast(self.currSession);
    _ = c.sqlite3_bind_int(stmt, 1, curS);

    // Evaluate the statement
    while (c.sqlite3_step(stmt) == c.SQLITE_ROW) {
        const hex_x = c.sqlite3_column_int(stmt, 0);
        const hex_y = c.sqlite3_column_int(stmt, 1);

        const aCity = terrain.WholeHex.init(hex_x, hex_y, 2);
        _ = try wh.append(allocator, aCity);
    }
}

// *****
pub fn add_map_forests(self: Database, allocator: std.mem.Allocator, wh: *std.ArrayList(terrain.WholeHex)) !void {
    // Prepare the SQL statement
    var stmt: ?*c.sqlite3_stmt = null;
    const sql =
        \\SELECT hex_x, hex_y FROM GameMap
        \\WHERE
        \\terrainNum = 4 AND
        \\sessionID = ?1
    ;
    _ = c.sqlite3_prepare_v2(self.db, sql, -1, &stmt, null);
    defer _ = c.sqlite3_finalize(stmt);

    // Binding
    const curS: i32 = @intCast(self.currSession);
    _ = c.sqlite3_bind_int(stmt, 1, curS);

    // Evaluate the statement
    while (c.sqlite3_step(stmt) == c.SQLITE_ROW) {
        const hex_x = c.sqlite3_column_int(stmt, 0);
        const hex_y = c.sqlite3_column_int(stmt, 1);

        const aForest = terrain.WholeHex.init(hex_x, hex_y, 4);
        _ = try wh.append(allocator, aForest);
    }
}

// *****
pub fn add_map_cultivated(self: Database, allocator: std.mem.Allocator, wh: *std.ArrayList(terrain.WholeHex)) !void {
    // Prepare the SQL statement
    var stmt: ?*c.sqlite3_stmt = null;
    const sql =
        \\SELECT hex_x, hex_y FROM GameMap
        \\WHERE
        \\terrainNum = 3 AND
        \\sessionID = ?1
    ;
    _ = c.sqlite3_prepare_v2(self.db, sql, -1, &stmt, null);
    defer _ = c.sqlite3_finalize(stmt);

    // Binding
    const curS: i32 = @intCast(self.currSession);
    _ = c.sqlite3_bind_int(stmt, 1, curS);
}

```

```

    // Evaluate the statement
    while (c.sqlite3_step(stmt) == c.SQLITE_ROW) {
        const hex_x = c.sqlite3_column_int(stmt, 0);
        const hex_y = c.sqlite3_column_int(stmt, 1);

        const aCultivated = terrain.WholeHex.init(hex_x, hex_y, 3);
        _ = try wh.append(allocator, aCultivated);
    }
}

// *****
pub fn add_map_rollings(self: Database, allocator: std.mem.Allocator, wh: *std.ArrayList(terrain.WholeHex)) !void {
    // Prepare the SQL statement
    var stmt: ?*c.sqlite3_stmt = null;
    const sql =
        \\SELECT hex_x, hex_y FROM GameMap
        \\WHERE
        \\terrainNum = 10 AND
        \\sessionID = ?1
    ;
    _ = c.sqlite3_prepare_v2(self.db, sql, -1, &stmt, null);
    defer _ = c.sqlite3_finalize(stmt);

    // Binding
    const curS: i32 = @intCast(self.currSession);
    _ = c.sqlite3_bind_int(stmt, 1, curS);

    // Evaluate the statement
    while (c.sqlite3_step(stmt) == c.SQLITE_ROW) {
        const hex_x = c.sqlite3_column_int(stmt, 0);
        const hex_y = c.sqlite3_column_int(stmt, 1);

        const aRolling = terrain.WholeHex.init(hex_x, hex_y, 10);
        _ = try wh.append(allocator, aRolling);
    }
}

// *****
pub fn add_map_hills(self: Database, allocator: std.mem.Allocator, hills: *std.ArrayList(terrain.Hill)) !void {
    // Prepare the SQL statement
    var stmt: ?*c.sqlite3_stmt = null;
    const sql =
        \\SELECT hex_x, hex_y, hex_z FROM GameMap
        \\WHERE
        \\(terrainNum = 5 or terrainNum = 6)
        \\AND sessionID = ?1
    ;
    _ = c.sqlite3_prepare_v2(self.db, sql, -1, &stmt, null);
    defer _ = c.sqlite3_finalize(stmt);

    // Binding
    const curS: i32 = @intCast(self.currSession);
    _ = c.sqlite3_bind_int(stmt, 1, curS);

    // Evaluate the statement
    while (c.sqlite3_step(stmt) == c.SQLITE_ROW) {
        const hex_x = c.sqlite3_column_int(stmt, 0);
        const hex_y = c.sqlite3_column_int(stmt, 1);
        const hex_z = c.sqlite3_column_int(stmt, 2);

        const aHill = terrain.Hill.init(hex_x, hex_y, hex_z);
        _ = try hills.append(allocator, aHill);
    }
}

// *****
pub fn add_map_bridges(self: Database, allocator: std.mem.Allocator, bridges: *std.ArrayList(terrain.Bridge)) !void {
    // Prepare the SQL statement
    var stmt: ?*c.sqlite3_stmt = null;
    const sql =
        \\SELECT hex_x, hex_y, spineLoc FROM GameMap
        \\WHERE
        \\terrainNum = 1 AND
        \\sessionID = ?1
    ;
    _ = c.sqlite3_prepare_v2(self.db, sql, -1, &stmt, null);
    defer _ = c.sqlite3_finalize(stmt);

    // Binding
    const curS: i32 = @intCast(self.currSession);
    _ = c.sqlite3_bind_int(stmt, 1, curS);

    // Evaluate the statement
    while (c.sqlite3_step(stmt) == c.SQLITE_ROW) {
        const hex_x = c.sqlite3_column_int(stmt, 0);
        const hex_y = c.sqlite3_column_int(stmt, 1);
        const spineLoc = c.sqlite3_column_int(stmt, 2);

        const aBridge = terrain.Bridge.init(hex_x, hex_y, spineLoc);
        _ = try bridges.append(allocator, aBridge);
    }
}

// *****
pub fn add_map_tunnel_entrances(self: Database, allocator: std.mem.Allocator, tunnel_entrances: *std.ArrayList(terrain.Tunnel_Entrance)) !void {
    // Prepare the SQL statement
    var stmt: ?*c.sqlite3_stmt = null;
    const sql =
        \\SELECT hex_x, hex_y, spineLoc FROM GameMap
        \\WHERE
        \\terrainNum = 14 AND
        \\sessionID = ?1

```

```

;
_ = c.sqlite3_prepare_v2(self.db, sql, -1, &stmt, null);
defer _ = c.sqlite3_finalize(stmt);

// Binding
const curS: i32 = @intCast(self.currSession);
_ = c.sqlite3_bind_int(stmt, 1, curS);

// Evaluate the statement
while (c.sqlite3_step(stmt) == c.SQLITE_ROW) {
    const hex_x = c.sqlite3_column_int(stmt, 0);
    const hex_y = c.sqlite3_column_int(stmt, 1);
    const spineLoc = c.sqlite3_column_int(stmt, 2);

    const aTE = terrain.Tunnel_Entrance.init(hex_x, hex_y, spineLoc);
    _ = try tunnel_entrances.append(allocator, aTE);
}
}

// *****
pub fn add_map_roads(self: Database, allocator: std.mem.Allocator, roads: *std.ArrayList(terrain.Road)) !void {
    // Prepare the SQL statement
    var stmt: ?*c.sqlite3_stmt = null;
    const sql =
        \\SELECT hex_x, hex_y, spineLoc FROM GameMap
        \\WHERE
        \\terrainNum = 9 AND
        \\sessionID = ?1
    ;
    _ = c.sqlite3_prepare_v2(self.db, sql, -1, &stmt, null);
    defer _ = c.sqlite3_finalize(stmt);

    // Binding
    const curS: i32 = @intCast(self.currSession);
    _ = c.sqlite3_bind_int(stmt, 1, curS);

    // Evaluate the statement
    while (c.sqlite3_step(stmt) == c.SQLITE_ROW) {
        const hex_x = c.sqlite3_column_int(stmt, 0);
        const hex_y = c.sqlite3_column_int(stmt, 1);
        const spineLoc = c.sqlite3_column_int(stmt, 2);

        const aRoad = terrain.Road.init(hex_x, hex_y, spineLoc);
        _ = try roads.append(allocator, aRoad);
    }
}

// *****
pub fn add_map_rivers(self: Database, allocator: std.mem.Allocator, rivers: *std.ArrayList(terrain.River)) !void {
    // Prepare the SQL statement
    var stmt: ?*c.sqlite3_stmt = null;
    const sql =
        \\SELECT hex_x, hex_y, spineLoc FROM GameMap
        \\WHERE
        \\terrainNum = 8 AND
        \\sessionID = ?1
    ;
    _ = c.sqlite3_prepare_v2(self.db, sql, -1, &stmt, null);
    defer _ = c.sqlite3_finalize(stmt);

    // Binding
    const curS: i32 = @intCast(self.currSession);
    _ = c.sqlite3_bind_int(stmt, 1, curS);

    // Evaluate the statement
    while (c.sqlite3_step(stmt) == c.SQLITE_ROW) {
        const hex_x = c.sqlite3_column_int(stmt, 0);
        const hex_y = c.sqlite3_column_int(stmt, 1);
        const spineLoc = c.sqlite3_column_int(stmt, 2);

        const aRiver = terrain.River.init(hex_x, hex_y, spineLoc);
        _ = try rivers.append(allocator, aRiver);
    }
}

// *****
pub fn add_map_lakes(self: Database, allocator: std.mem.Allocator, lakes: *std.ArrayList(terrain.Lake)) !void {
    // Prepare the SQL statement
    var stmt: ?*c.sqlite3_stmt = null;
    const sql =
        \\SELECT hex_x, hex_y FROM GameMap
        \\WHERE
        \\terrainNum = 7 AND
        \\sessionID = ?1
    ;
    _ = c.sqlite3_prepare_v2(self.db, sql, -1, &stmt, null);
    defer _ = c.sqlite3_finalize(stmt);

    // Binding
    const curS: i32 = @intCast(self.currSession);
    _ = c.sqlite3_bind_int(stmt, 1, curS);

    // Evaluate the statement
    while (c.sqlite3_step(stmt) == c.SQLITE_ROW) {
        const hex_x = c.sqlite3_column_int(stmt, 0);
        const hex_y = c.sqlite3_column_int(stmt, 1);

        const aLake = terrain.Lake.init(hex_x, hex_y);
        _ = try lakes.append(allocator, aLake);
    }
}

// *****

```

```

pub fn add_map_tiles(self: Database, allocator: std.mem.Allocator, textures: *std.ArrayList(r1.Texture), tiles: *std.ArrayList(tile.Tile)) !
void {
    // Prepare the SQL statement
    var stmt: ?*c.sqlite3_stmt = null;
    const sql =
        \\SELECT val_text FROM GameMeta
        \\WHERE
        \\attrib = 'tiles' AND
        \\sessionID = ?1
    ;
    _ = c.sqlite3_prepare_v2(self.db, sql, -1, &stmt, null);
    defer _ = c.sqlite3_finalize(stmt);

    // Binding
    const curS: i32 = @intCast(self.currSession);
    _ = c.sqlite3_bind_int(stmt, 1, curS);

    // Evaluate the statement
    if (c.sqlite3_step(stmt) != c.SQLITE_ROW) return;

    const raw_str = c.sqlite3_column_text(stmt, 0);
    const tileLetters: []const u8 = std.mem.span(raw_str);

    // =====
    if (std.mem.containsAtLeast(u8, tileLetters, 1, "A") or std.mem.containsAtLeast(u8, tileLetters, 1, "a")) {
        // add image to asset collection
        const index = textures.items.len;
        try textures.append(allocator, try r1.loadTexture("TLR/Map A.png"));

        // add tiles
        if (std.mem.containsAtLeast(u8, tileLetters, 1, "A")) {
            const tl = tile.Tile.init(index, 0, 0.0);
            _ = try tiles.append(allocator, tl);
        }

        if (std.mem.containsAtLeast(u8, tileLetters, 1, "a")) {
            const tl = tile.Tile.init(index, 0, 180.0);
            _ = try tiles.append(allocator, tl);
        }
    }
    if (std.mem.containsAtLeast(u8, tileLetters, 1, "B") or std.mem.containsAtLeast(u8, tileLetters, 1, "b")) {
        // add image to asset collection
        const index = textures.items.len;
        try textures.append(allocator, try r1.loadTexture("TLR/Map B.png"));

        // add tiles
        if (std.mem.containsAtLeast(u8, tileLetters, 1, "B")) {
            const tl = tile.Tile.init(index, 1, 0.0);
            _ = try tiles.append(allocator, tl);
        }

        if (std.mem.containsAtLeast(u8, tileLetters, 1, "b")) {
            const tl = tile.Tile.init(index, 1, 180.0);
            _ = try tiles.append(allocator, tl);
        }
    }
    if (std.mem.containsAtLeast(u8, tileLetters, 1, "C") or std.mem.containsAtLeast(u8, tileLetters, 1, "c")) {
        // add image to asset collection
        const index = textures.items.len;
        try textures.append(allocator, try r1.loadTexture("TLR/Map C.png"));

        // add tiles
        if (std.mem.containsAtLeast(u8, tileLetters, 1, "C")) {
            const tl = tile.Tile.init(index, 2, 0.0);
            _ = try tiles.append(allocator, tl);
        }

        if (std.mem.containsAtLeast(u8, tileLetters, 1, "c")) {
            const tl = tile.Tile.init(index, 2, 180.0);
            _ = try tiles.append(allocator, tl);
        }
    }
    if (std.mem.containsAtLeast(u8, tileLetters, 1, "D") or std.mem.containsAtLeast(u8, tileLetters, 1, "d")) {
        // add image to asset collection
        const index = textures.items.len;
        try textures.append(allocator, try r1.loadTexture("TLR/Map D.png"));

        // add tiles
        if (std.mem.containsAtLeast(u8, tileLetters, 1, "D")) {
            const tl = tile.Tile.init(index, 3, 0.0);
            _ = try tiles.append(allocator, tl);
        }

        if (std.mem.containsAtLeast(u8, tileLetters, 1, "d")) {
            const tl = tile.Tile.init(index, 3, 180.0);
            _ = try tiles.append(allocator, tl);
        }
    }
    if (std.mem.containsAtLeast(u8, tileLetters, 1, "E") or std.mem.containsAtLeast(u8, tileLetters, 1, "e")) {
        // add image to asset collection
        const index = textures.items.len;
        try textures.append(allocator, try r1.loadTexture("TLR/Map E.png"));

        // add tiles
        if (std.mem.containsAtLeast(u8, tileLetters, 1, "E")) {
            const tl = tile.Tile.init(index, 4, 0.0);
            _ = try tiles.append(allocator, tl);
        }

        if (std.mem.containsAtLeast(u8, tileLetters, 1, "e")) {
            const tl = tile.Tile.init(index, 4, 180.0);
            _ = try tiles.append(allocator, tl);
        }
    }
}

```

```

}

// *****
pub fn get_tileLetters(self: Database, allocator: std.mem.Allocator) ![]u8 {
    // Prepare the SQL statement
    var stmt: ?*c.sqlite3_stmt = null;
    const sql =
        \\SELECT val_text FROM GameMeta
        \\WHERE
        \\attrib = 'tiles' AND
        \\sessionID = ?1
    ;
    _ = c.sqlite3_prepare_v2(self.db, sql, -1, &stmt, null);
    defer _ = c.sqlite3_finalize(stmt);

    // Binding
    const curS: i32 = @intCast(self.currSession);
    _ = c.sqlite3_bind_int(stmt, 1, curS);

    // Evaluate the statement
    if (c.sqlite3_step(stmt) != c.SQLITE_ROW) return undefined;

    const raw_str = c.sqlite3_column_text(stmt, 0);

    return try std.fmt.allocPrint(allocator, "{s}", .{raw_str});
}

// *****
pub fn get_int_vals_session(self: Database, attrib: []const u8) struct { i32, i32, i32 } {
    // Prepare statement
    const query1 =
        \\SELECT val_int0, val_int1, val_int2 FROM GameMeta
        \\WHERE
        \\ attrib = ?1
        \\ AND
        \\ sessionID = ?2
    ;
    var stmt: ?*c.sqlite3_stmt = null;

    if (c.sqlite3_prepare_v2(self.db, query1, -1, &stmt, null) != c.SQLITE_OK) {
        print("Failed to prepare statement(1): {s}\n", .{c.sqlite3_errmsg(self.db)});
    }
    defer _ = c.sqlite3_finalize(stmt);

    // Binding
    _ = c.sqlite3_bind_text(stmt, 1, attrib.ptr, @intCast(attrib.len), c.SQLITE_TRANSIENT);
    const curS: i32 = @intCast(self.currSession);
    _ = c.sqlite3_bind_int(stmt, 2, curS);

    // Evaluate the statement
    if (c.sqlite3_step(stmt) == c.SQLITE_ROW) {
        const x: i32 = c.sqlite3_column_int(stmt, 0);
        const y: i32 = c.sqlite3_column_int(stmt, 1);
        const z: i32 = c.sqlite3_column_int(stmt, 2);

        return .{ x, y, z };
    }

    return .{ 0, 0, 0 };
}

// *****
pub fn get_float_vals(self: Database, attrib: []const u8) struct { f32, f32, f32 } {
    // Prepare statement
    const query1 =
        \\SELECT val_real0, val_real1, val_real2 FROM GameMeta
        \\WHERE
        \\ attrib = ?1
        \\ AND
        \\ sessionID = 0
    ;
    var stmt: ?*c.sqlite3_stmt = null;

    if (c.sqlite3_prepare_v2(self.db, query1, -1, &stmt, null) != c.SQLITE_OK) {
        print("Failed to prepare statement(1): {s}\n", .{c.sqlite3_errmsg(self.db)});
    }
    defer _ = c.sqlite3_finalize(stmt);

    // Binding
    _ = c.sqlite3_bind_text(stmt, 1, attrib.ptr, @intCast(attrib.len), c.SQLITE_TRANSIENT);
    // const curS: i32 = @intCast(self.currSession);
    // _ = c.sqlite3_bind_int(stmt, 2, curS);

    // Evaluate the statement
    if (c.sqlite3_step(stmt) == c.SQLITE_ROW) {
        const x: f32 = @floatCast(c.sqlite3_column_double(stmt, 0));
        const y: f32 = @floatCast(c.sqlite3_column_double(stmt, 1));
        const z: f32 = @floatCast(c.sqlite3_column_double(stmt, 2));
        return .{ x, y, z };
    }

    return .{ 0.0, 0.0, 0.0 };
}

// *****
pub fn foo(self: Database) void {
    print("{d}\n", .{self.currSession});
    print("{d},{d}\n", .{ self.pixelCount[0], self.pixelCount[1] });
}

// *****
pub fn close(self: Database) void {

```

```

    _ = c.sqlite3_close(self.db);
}

// *****
pub fn get_Point(self: Database, attrib: []const u8) struct { i32, i32 } {
    // Prepare statement
    const query1 =
        \\SELECT val_int0, val_int1 FROM GameMeta
        \\WHERE
        \\    attrib = ?1
        \\    AND
        \\    sessionID = 0
    ;

    var stmt: ?*c.sqlite3_stmt = null;

    if (c.sqlite3_prepare_v2(self.db, query1, -1, &stmt, null) != c.SQLITE_OK) {
        print("Failed to prepare statement(1): {s}\n", .{c.sqlite3_errmsg(self.db)});
    }
    defer _ = c.sqlite3_finalize(stmt);

    // Binding
    _ = c.sqlite3_bind_text(stmt, 1, attrib.ptr, @intCast(attrib.len), c.SQLITE_TRANSIENT);

    // Evaluate the statement
    if (c.sqlite3_step(stmt) == c.SQLITE_ROW) {
        const x: i32 = @intCast(c.sqlite3_column_int64(stmt, 0));
        const y: i32 = @intCast(c.sqlite3_column_int64(stmt, 1));
        return .{ x, y };
    }

    return .{ 0, 0 };
}
};

***** GameMap.zig
const std = @import("std");
const print = std.debug.print;

const tile = @import("Tile.zig");

pub const GameMap = struct {
    GMap: [4][4]i32,

    pub fn init(tileLetters: []const u8, tiles: *std.ArrayList(tile.Tile)) GameMap {
        var GMap: [4][4]i32 = [4][4]i32{
            [4]i32{ -1, -1, -1, -1 },
            [4]i32{ -1, -1, -1, -1 },
            [4]i32{ -1, -1, -1, -1 },
            [4]i32{ -1, -1, -1, -1 },
        };

        var it_Row = std.mem.splitScalar(u8, tileLetters, '_');
        var locCR: [2]i32 = .{ 0, 0 };
        while (it_Row.next()) |Row| {
            var it_Col = std.mem.splitScalar(u8, Row, ',');
            while (it_Col.next()) |Col| {
                if (Col.len < 1) {
                    locCR[0] += 1;
                    continue;
                }
                GMap[@intCast(locCR[0])][@intCast(locCR[1])] = @intCast(Col[0]);
                locCR[0] += 1;
            }
            locCR[0] = 0;
            locCR[1] += 1;
        }

        for (0..4) |row| {
            for (0..4) |col| {
                const X = get_tile_id(tiles, GMap[@intCast(col)][@intCast(row)]);
                GMap[@intCast(col)][@intCast(row)] = X;
            }
        }

        return GameMap{
            .GMap = GMap,
        };
    }
}

// *****
fn get_tile_id(tiles: *std.ArrayList(tile.Tile), letter: i32) i32 {
    const ref_A: i32 = 'A';
    const ref_a: i32 = 'a';

    var rotation: f32 = 0.0;
    var ref = letter - ref_A;
    if (ref > 26) {
        ref = letter - ref_a;
        rotation = 180;
    }

    var tile_id: i32 = -1;
    for (0..tiles.items.len) |x| {
        if (ref >= 0) {
            if ((ref == tiles.items[x].id) and (rotation == tiles.items[x].rotation))
                tile_id = @intCast(x);
        }
    }

    return tile_id;
}

```

```

// *****
pub fn status(self: GameMap, tiles: *std.ArrayList(tile.Tile)) void {
    for (0..tiles.items.len) |i| {
        print("id:{d} --- index:{d}\n", .{ i, tiles.items.ptr[i].index });
    }
    for (0..4) |row| {
        for (0..4) |col| {
            const tl = self.GMap[col][row];
            print("({d},{d}).{d} ", .{ col, row, tl });
        }
        print("\n", .{});
    }
}

pub fn bar(self: GameMap) void {
    _ = self;
    print("***** yeah *****\n", .{});
}

};

***** Hexagon.zig

const std = @import("std");
const print = std.debug.print;

const terrain = @import("Terrain.zig");

const cos30: f64 = 0.86602540378; // degree base

pub const Hexagon = struct {
    x: i32,
    y: i32,
    cartX: f64,
    cartY: f64,

    pub fn init(x: i32, y: i32) Hexagon {
        const X: f64 = x * cos30;
        var Y: f64 = 0.0;
        if (@mod(x, 2) > 0)
            Y = @as(f64, @floatFromInt(y)) - 0.5
        else
            Y = @as(f64, @floatFromInt(y));

        return Hexagon{
            .x = x,
            .y = y,
            .cartX = X,
            .cartY = Y,
        };
    }
} // *****

pub fn get_hex_distance(self: Hexagon, allocator: std.mem.Allocator, target: Hexagon, ph: *std.ArrayList(terrain.Path)) !i32 {
    try get_path(self, allocator, target, ph);
    var dist: i32 = @intCast(ph.items.len);
    dist -= 1;
    return dist;
}

// *****
pub fn get_path(self: Hexagon, allocator: std.mem.Allocator, target: Hexagon, ph: *std.ArrayList(terrain.Path)) !void {
    ph.clearAndFree(allocator);

    if (self.x == target.x) {
        if (self.y == target.y) {
            print("same\n", .{});
            return;
        }
    }

    const aPath = terrain.Path.init(self.x, self.y);
    _ = try ph.append(allocator, aPath);

    const refAngle: f64 = self.degrees(target);

    const hexDir = get_hexDir(self, degrees(self, target));
    var adjDirs = [_]i32{ 0, 0, 0 };
    adjDirs[1] = hexDir;
    adjDirs[0] = hexDir - 1;
    if (adjDirs[0] < 0) adjDirs[0] = 5;
    adjDirs[2] = hexDir + 1;
    if (adjDirs[2] > 5) adjDirs[2] = 0;
    try the_path(allocator, self, target, adjDirs, refAngle, ph);
}

// -----
pub fn the_path(allocator: std.mem.Allocator, wlk: Hexagon, target: Hexagon, adjDirs: [3]i32, refAngle: f64, ph:
*std.ArrayList(terrain.Path)) !void {
    if (wlc.x == target.x) {
        if (wlc.y == target.y) {
            print("same\n", .{});
            return;
        }
    }

    const hexLoc = get_adj_hexLoc(wlk, adjDirs[0]);
    if (hexLoc[0] == target.x) {
        if (hexLoc[1] == target.y) {
            print("same\n", .{});
            return;
        }
    }

    var aHexagon = Hexagon.init(hexLoc[0], hexLoc[1]);
    var angle = @abs(aHexagon.degrees(target) - refAngle);
    for (1..3) |i| {
        const hexLoc1 = get_adj_hexLoc(wlk, adjDirs[i]);

```

```

        if (hexLoc1[0] == target.x) {
            if (hexLoc1[1] == target.y) {
                const aPath = terrain.Path.init(target.x, target.y);
                _ = try ph.append(allocator, aPath);
                return;
            }
        }
        const aHexagon1 = Hexagon.init(hexLoc1[0], hexLoc1[1]);
        const angle1 = @abs(aHexagon1.degrees(target) - refAngle);
        if (angle > angle1) {
            aHexagon = aHexagon1;
            angle = angle1;
        }
    }

    const aPath = terrain.Path.init(aHexagon.x, aHexagon.y);
    _ = try ph.append(allocator, aPath);
    try the_path(allocator, aHexagon, target, adjDirs, refAngle, ph); // recursing
}

// *****
pub fn get_hexDir(self: Hexagon, dir: f64) i32 {
    _ = self;
    const d: i32 = @as(i32, @intFromFloat(dir));
    const direction: i32 = switch (d) {
        0...59 => 1,
        60...119 => 0,
        120...179 => 5,
        180...239 => 4,
        240...299 => 3,
        else => 2,
    };
    return direction;
}

// *****
pub fn get_adj_hexLoc(self: Hexagon, hexDir: i32) struct { i32, i32 } {
    var x = self.x;
    var y = self.y;

    if (@mod(self.x, 2) == 0) {
        switch (hexDir) {
            0 => y -= 1,
            1 => x += 1,
            2 => {
                x += 1;
                y += 1;
            },
            3 => y += 1,
            4 => {
                x -= 1;
                y += 1;
            },
            else => x -= 1,
        }
        return .{ x, y };
    }

    switch (hexDir) {
        0 => y -= 1,
        1 => {
            x += 1;
            y -= 1;
        },
        2 => x += 1,
        3 => y += 1,
        4 => x -= 1,
        else => {
            x -= 1;
            y -= 1;
        },
    },
    return .{ x, y };
}

pub fn degrees(self: Hexagon, other: Hexagon) f64 {
    const deltaX: f64 = other.cartX - self.cartX;
    const deltaY: f64 = other.cartY - self.cartY;
    const angle_rad = std.math.atan2(deltaY, deltaX);
    var deg = 360.0 - std.math.radiansToDegrees(angle_rad);
    if (deg >= 360.0) deg -= 360.0;
    return deg;
}

pub fn cartesian_dist(self: Hexagon, other: Hexagon) f64 {
    const deltaX: f64 = @abs(self.cartX - other.cartX);
    const deltaY: f64 = @abs(self.cartY - other.cartY);
    return std.math.sqrt((deltaX * deltaX) + (deltaY * deltaY));
}
};

***** Terrain.zig
const rl = @import("raylib");
const std = @import("std");
const print = std.debug.print;

pub const light_brown = rl.Color.init(0xA6, 0x9A, 0x8E, 0xFF);
pub const pale_yellow = rl.Color.init(0xCE, 0xCC, 0xBF, 0xFF);

pub const Path = struct {
    x: i32,
    y: i32,

```

```

f: u32, // flags

pub fn init(x: i32, y: i32) Path {
    return Path{
        .x = x,
        .y = y,
        .f = 0,
    };
}
// bit 0: blocks LOS
// bit 1:
};

pub const Hill = struct {
    x: i32,
    y: i32,
    h: i32, // height

    pub fn init(x: i32, y: i32, h: i32) Hill {
        return Hill{
            .x = x,
            .y = y,
            .h = h,
        };
    }
};

pub const Lake = struct {
    x: i32,
    y: i32,

    pub fn init(x: i32, y: i32) Lake {
        return Lake{
            .x = x,
            .y = y,
        };
    }
};

pub const WholeHex = struct {
    x: i32,
    y: i32,
    id: i32,

    pub fn init(x: i32, y: i32, id: i32) WholeHex {
        return WholeHex{
            .x = x,
            .y = y,
            .id = id,
        };
    }
};

// id (terrainNum)
// *****
// 10: Rolling
// 3: Cultivated

pub const River = struct {
    x: i32,
    y: i32,
    s: i32, // spineLoc

    pub fn init(x: i32, y: i32, s: i32) River {
        return River{
            .x = x,
            .y = y,
            .s = s,
        };
    }
};

pub const Road = struct {
    x: i32,
    y: i32,
    s: i32, // spineLoc

    pub fn init(x: i32, y: i32, s: i32) Road {
        return Road{
            .x = x,
            .y = y,
            .s = s,
        };
    }
};

pub const Bridge = struct {
    x: i32,
    y: i32,
    s: i32, // spineLoc

    pub fn init(x: i32, y: i32, s: i32) Bridge {
        return Bridge{
            .x = x,
            .y = y,
            .s = s,
        };
    }
};

pub const Tunnel_Entrance = struct {
    x: i32,
    y: i32,
    s: i32, // spineLoc

```

```

pub fn init(x: i32, y: i32, s: i32) Tunnel_Entrance {
    return Tunnel_Entrance{
        .x = x,
        .y = y,
        .s = s,
    };
}

// *****
pub fn is_hill_blocks_LOS(h: *std.ArrayList(Hill), p: *std.ArrayList(Path)) bool {
    for (0..h.items.len) |i| {
        for (0..p.items.len) |j| {
            if (h.items.ptr[i].x == p.items.ptr[j].x) {
                if (h.items.ptr[i].y == p.items.ptr[j].y) return true;
            }
        }
    }
    return false;
}

// *****
pub fn get_3rd_pt(pt1: struct { i32, i32 }, pt2: struct { i32, i32 }) struct { i32, i32 } {
    return .{ pt1[0] + pt2[0], pt1[1] + pt2[1] };
}

```

***** Tile.zig

```

const r1 = @import("raylib");
const std = @import("std");
const print = std.debug.print;

pub const Tile = struct {
    index: usize, // position in the arena
    id: usize, // map ID
    rotation: f32,

    // *****
    pub fn init(index: usize, id: usize, rotation: f32) Tile {
        return Tile{
            .index = index,
            .id = id,
            .rotation = rotation,
        };
    }
};

// id
// *****
// 0: map A
// 1: map B
// 2: map C
// 3: map D
// 4: map E

```

***** UI.zig

```

const r1 = @import("raylib");
const std = @import("std");
const print = std.debug.print;
const asset = @import("Asset.zig");

var mousePos: r1.Vector2 = undefined;

pub fn mode_1(GameFlags: *u64, camera: r1.Camera2D, hex_width: f32, hex_height: f32, cmb: *std.ArrayList(asset.Combatant), cuih:
*std.ArrayList(asset.Combatant), allocator: std.mem.Allocator) !void {
    print("yo: {d}\n", .{GameFlags.*});

    const screenMousePos = r1.getMousePosition();
    const worldMousePos = r1.getScreenToWorld2D(screenMousePos, camera);
    mousePos = screenMousePos;

    var mouseX = worldMousePos.x - 101.0;
    mouseX /= hex_width;

    const X: i32 = @ceil(mouseX);
    var Y: i32 = 0;

    if (@mod(X, 2) == 0) {
        var mouseY = worldMousePos.y - 116.79166;
        mouseY /= hex_height;
        Y = @ceil(mouseY);
    } else {
        var mouseY = worldMousePos.y;
        mouseY /= hex_height;
        Y = @ceil(mouseY);
    }

    //print("{d},{d}\n\n", .{ X, Y });
    //GameFlags_prev.* = GameFlags.*;
    GameFlags.* ^= (1 << 0);

    for (0..cmb.items.len) |i| {
        if (cmb.items.ptr[i].hex_x == X) {
            if (cmb.items.ptr[i].hex_y == Y) {
                if ((GameFlags.* & (1 << 1)) == 0) { // choose a hex with units
                    print("{d},{d}: {s}\n", .{ cmb.items.ptr[i].hex_x, cmb.items.ptr[i].hex_y, cmb.items.ptr[i].descrip });
                    GameFlags.* ^= (1 << 1);

                    _ = try cuih.append(allocator, cmb.items.ptr[i]);
                    break;
                }
            }
        }
    }
}

```

```
    }  
  }  
  print("no units\n", .{});  
}
```